

# Attendance

5bg73t



# Computer Organisation

---

## **Topic 5**

## **Addressing, and Instruction Set Characteristics & Functions**

Hamacher et al., Computer Organisation (5th Ed.), Chap. 2

Stallings, Computer Organization & Architecture (9th Ed.), Chap. 12

# Outline

---

- Memory locations and addresses
  - Byte addressability
  - Big-endian and Little-endian Assignments
- Memory operations
- Instructions and instruction sequencing
  - Notation
  - Types of instruction
  - Types of operands
  - Types of operations

# Memory Locations

---

- **Operands** and **instructions** are stored in **memory**

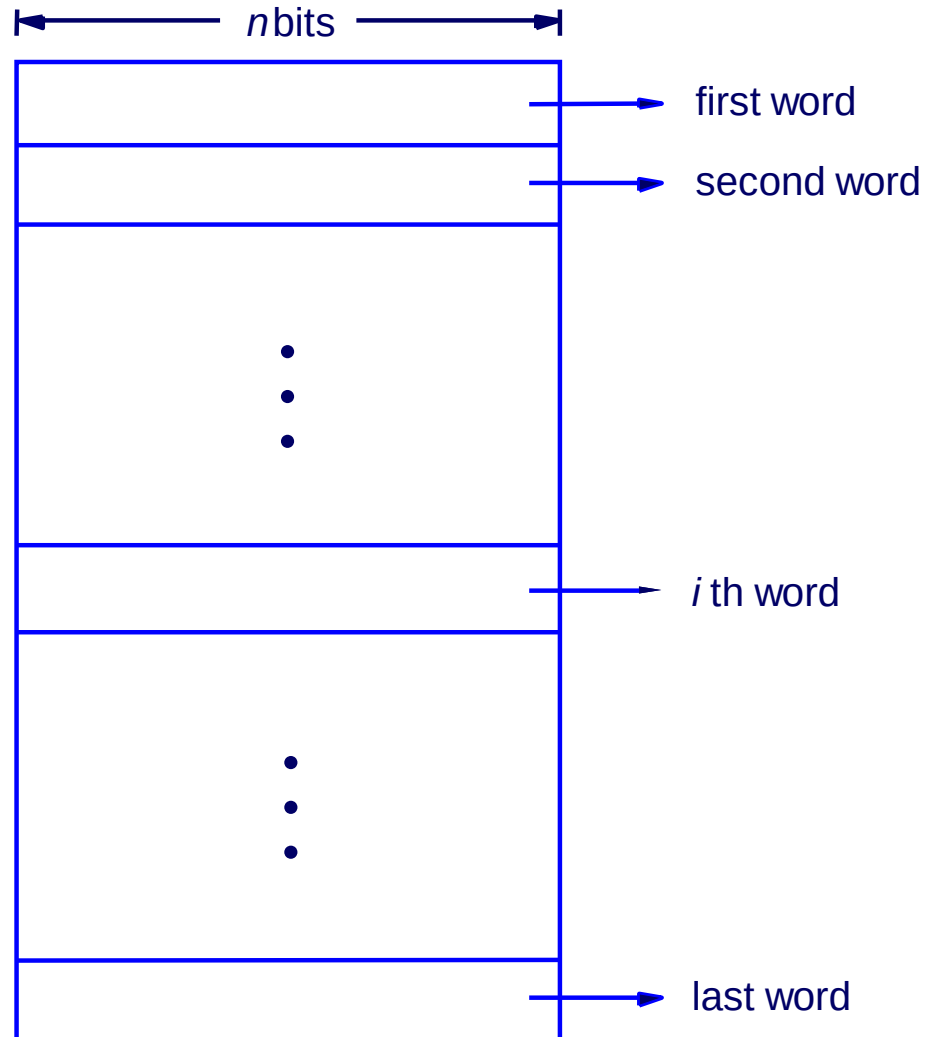
# Memory Locations

---

- Memory consists of many millions of storage cells
  - Each cell can store a “bit” with value 0 or 1
- Each group of  $n$  bits is called a “word”
  - $n$  = word length

# Memory Locations

---



Memory words

# Memory Locations

---

- Many computers have word lengths from 16-64 bits
- **A unit of 8 bits = 1 “byte”**

# Memory Locations

---

- Memory location contains
  - **Instruction** or **Operand**
  - Operand can be **number/integer** or **character**



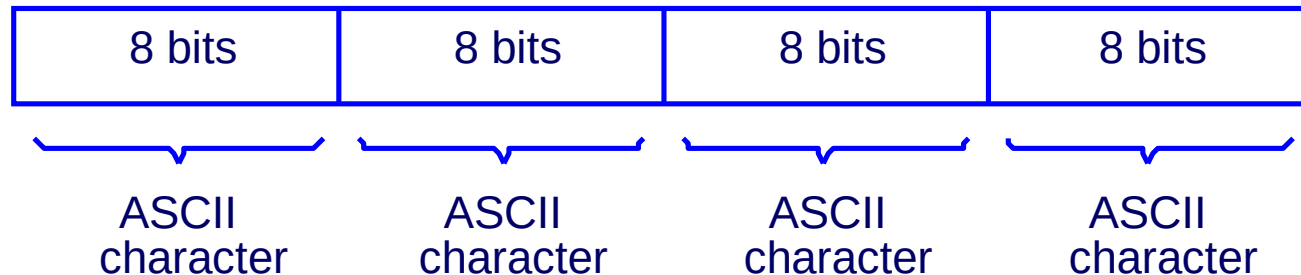
# Memory Locations

---



↑ Sign bit:  $b_{31} = 0$  for positive numbers  
 $b_{31} = 1$  for negative numbers

(a) A signed integer



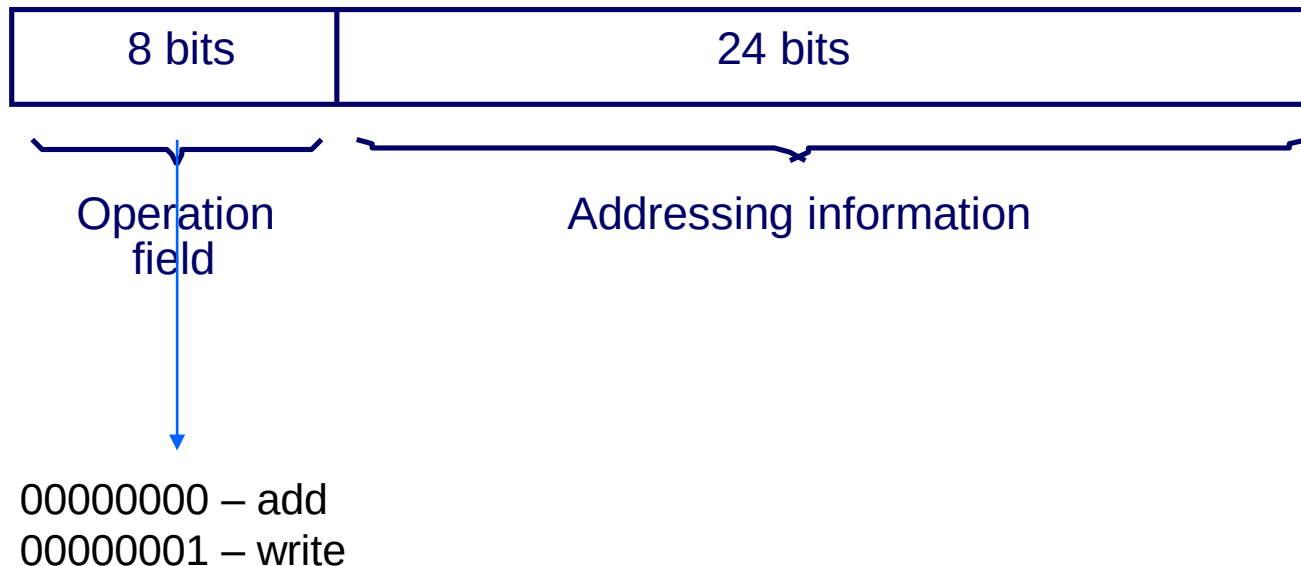
(b) Four characters

Examples of encoded information in a 32-bit word

# Memory Locations

---

- For instructions, memory word has **fields** for
  - Operation
  - Operand



# Memory Locations

---

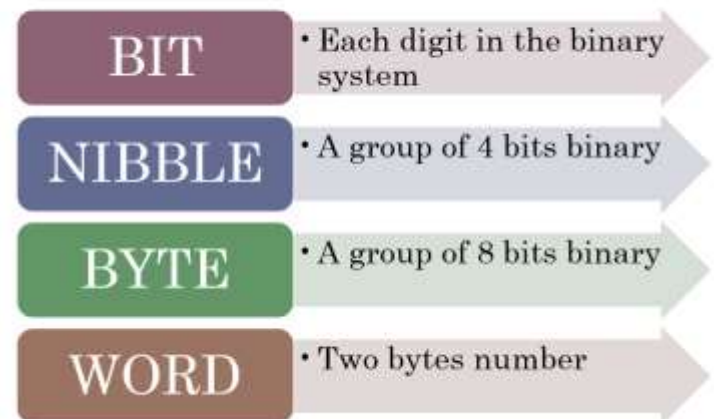
- **Accessing** memory to store/retrieve requires **addresses**
  - Addresses expressed from 0 through  $2^k - 1$
  - $2^k$  **addresses** constitute the **address space** of the computer,  $2^k$  addressable locations

# Byte Addressability

---

- Not practical for each bit to have a memory address
- Practical to have successive addresses for **successive byte locations** = byte addressability

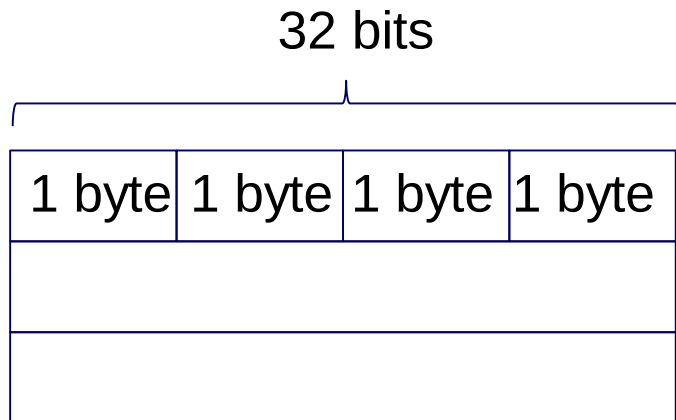
## The terminology



# Byte Addressability

---

- Example
  - If byte location have addresses 0, 1, 2, ...
  - If word length = 32 bits
  - Then successive words are located at addresses 0, 4, 8, ... (each word 4 bytes)



|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 |
| 4 | 4 | 5 | 6 | 7 |
| 8 |   |   |   |   |
|   |   |   |   |   |

# Quick Activity #1

---

What is the successive byte location for the following word length:

- 1) 16 bits
- 2) 64 bits
- 3) 128 bits?

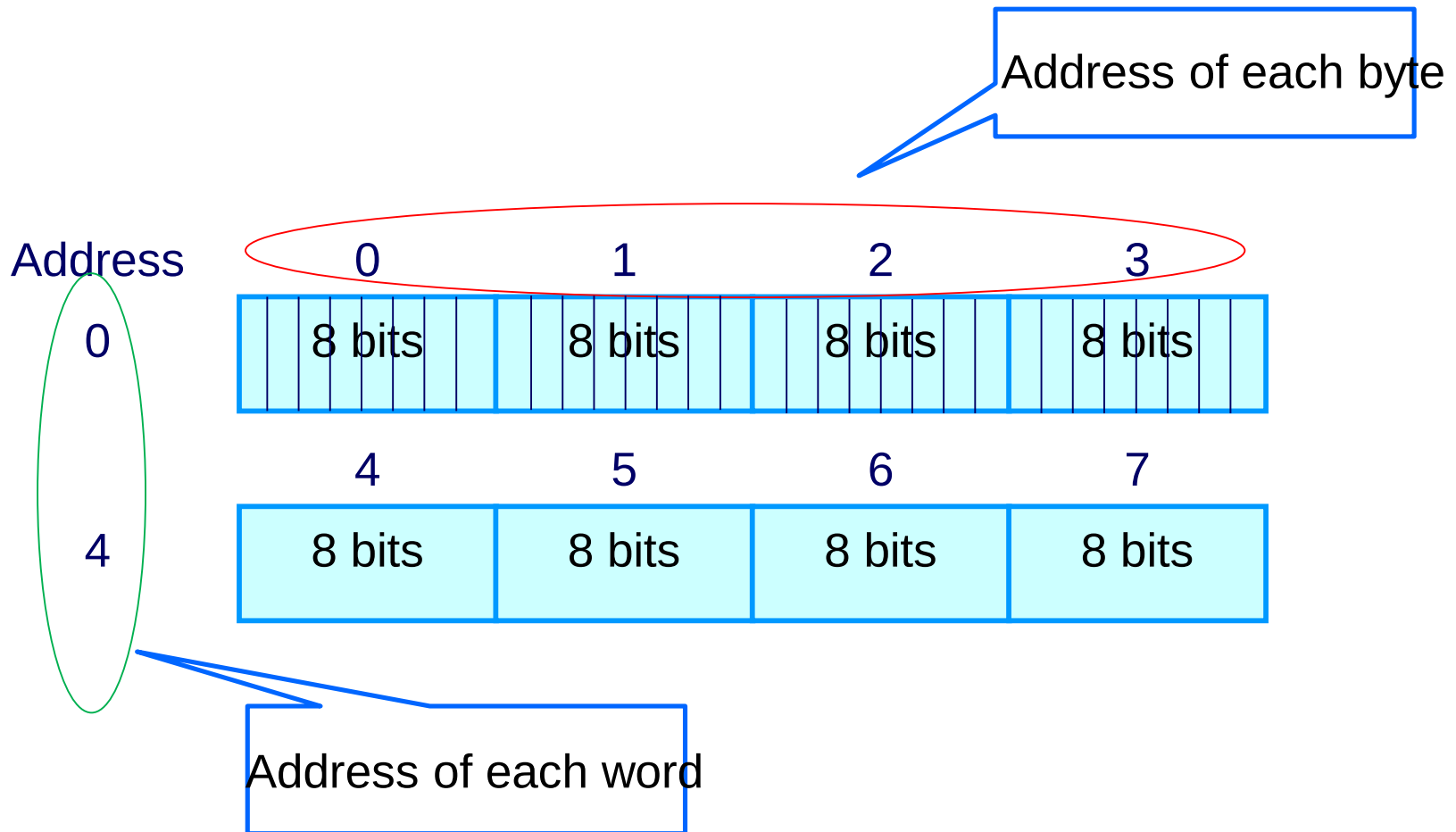
16 bits

|   |                                                                                   |   |
|---|-----------------------------------------------------------------------------------|---|
|   |  |   |
| 0 | 0                                                                                 | 1 |
| 2 | 2                                                                                 | 3 |
| 4 | 4                                                                                 | 5 |
| 6 | 6                                                                                 | 7 |
|   |                                                                                   |   |

64 bits

|    |                                                                                     |    |    |    |    |    |    |    |
|----|-------------------------------------------------------------------------------------|----|----|----|----|----|----|----|
|    |  |    |    |    |    |    |    |    |
| 0  | 0                                                                                   | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 8  | 8                                                                                   | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
| 16 | 16                                                                                  | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 24 |                                                                                     |    |    |    |    |    |    |    |
|    |                                                                                     |    |    |    |    |    |    |    |

# Byte Addressability



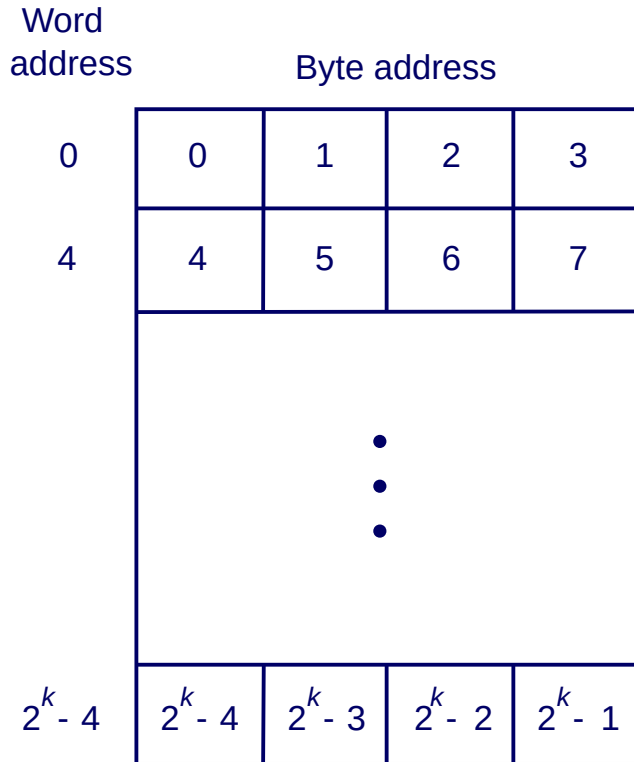
# Big-endian and Little endian

---

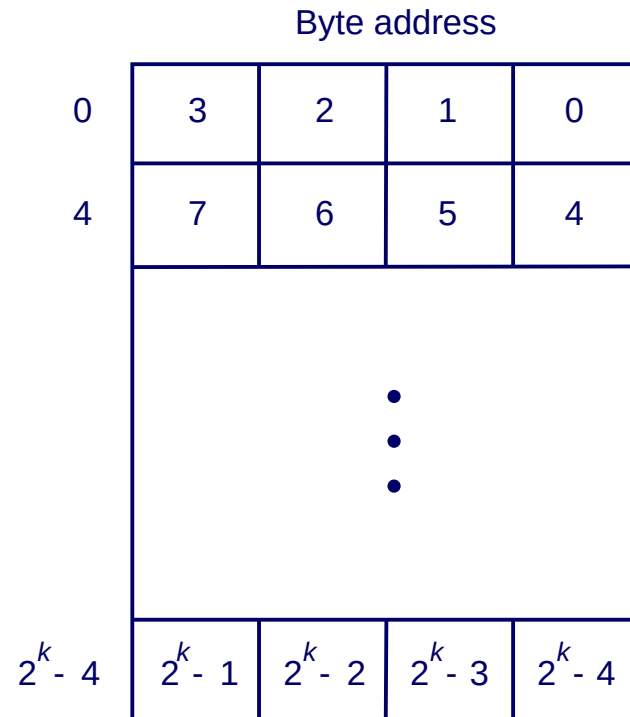
- Two ways that byte addresses can be assigned
- **Big-endian**
  - When lower byte addresses are used for the more significant (leftmost) bytes
- **Little-endian**
  - When lower byte addresses are used for the less significant (rightmost) bytes
- Both are used in commercial machines



# Big-endian and Little endian



(a) Big-endian assignment



(b) Little-endian assignment

# Big-endian and Little endian

---

```
struct {  
    int        a;           //0x1112_1314  
    int        pad;        //  
    double     b;           //0x2122_2324_2526_2728  
    char*      c;           //0x3132_3334  
    char       d[7];        //A,B,C,D,E,F,G  
    short      e;           //0x5152  
    int        f;           //0x6162_6364  
} s;
```

Int a;

Int pad;

# Big-endian and Little endian

Big-endian address mapping

| Byte address | 00 |    |    |    |    |    |    |    |
|--------------|----|----|----|----|----|----|----|----|
|              | 11 | 12 | 13 | 14 | 04 | 05 | 06 | 07 |
| 08           | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 10           | 31 | 32 | 33 | 34 | A  | B  | C  | D  |
| 18           | E  | F  | G  |    | 51 | 52 |    |    |
| 20           | 61 | 62 | 63 | 64 |    |    |    |    |

Little-endian address mapping

| Byte address | 00 |    |    |    |    |    |    |    |
|--------------|----|----|----|----|----|----|----|----|
|              | 07 | 06 | 05 | 04 | 11 | 12 | 13 | 14 |
| 08           | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 |
| 10           | D  | C  | B  | A  | 31 | 32 | 33 | 34 |
| 18           |    |    | 51 | 52 |    | G  | F  | E  |
| 20           |    |    | 61 | 62 | 63 | 64 |    |    |

# Word Alignment

---

- Words are aligned when they begin at a byte address that is a multiple of the number of bytes
- Number of bytes in a word is a power of 2
- Example
  - If word length is 16 bits ( $2^1$  bytes), aligned words begin at byte addresses 0, 2, 4, ...
  - If word length is 32 bits ( $2^2$  bytes), aligned words begin at byte addresses 0, 4, 8, ...

# Outline

---

- Memory locations and addresses
  - Byte addressability
  - Big-endian and Little-endian Assignments
- Memory operations
- Instructions and instruction sequencing
  - Notation
  - Types of instruction
  - Types of operands
  - Types of operations

# Accessing Memory

---

- **Numbers/characters** usually occupy  
**1 word**

# Accessing Memory

---

- For **character strings**
  - Beginning of string indicated by giving **byte address of first character**
  - Successive byte locations contain successive characters of the string
  - **End of string** indicated by
    - Special control character to mean “end of string”
    - Separate memory word location or processor register to store number indicating string length

# Memory Operations

---

- To execute instructions...
- Word(s) containing **instruction** need to be transferred **from memory to processor**
- **Operands/results** also need to be moved **between memory and processor**



# Memory Operations

---

- Two basic operations:
  - o **Load** (Read/Fetch)
  - o **Store** (Write)

# Memory Operations

---

- **Load**
  - **Processor sends address** of memory location and **requests content be read**
  - **Memory reads data** at that address and **sends to processor**

# Memory Operations

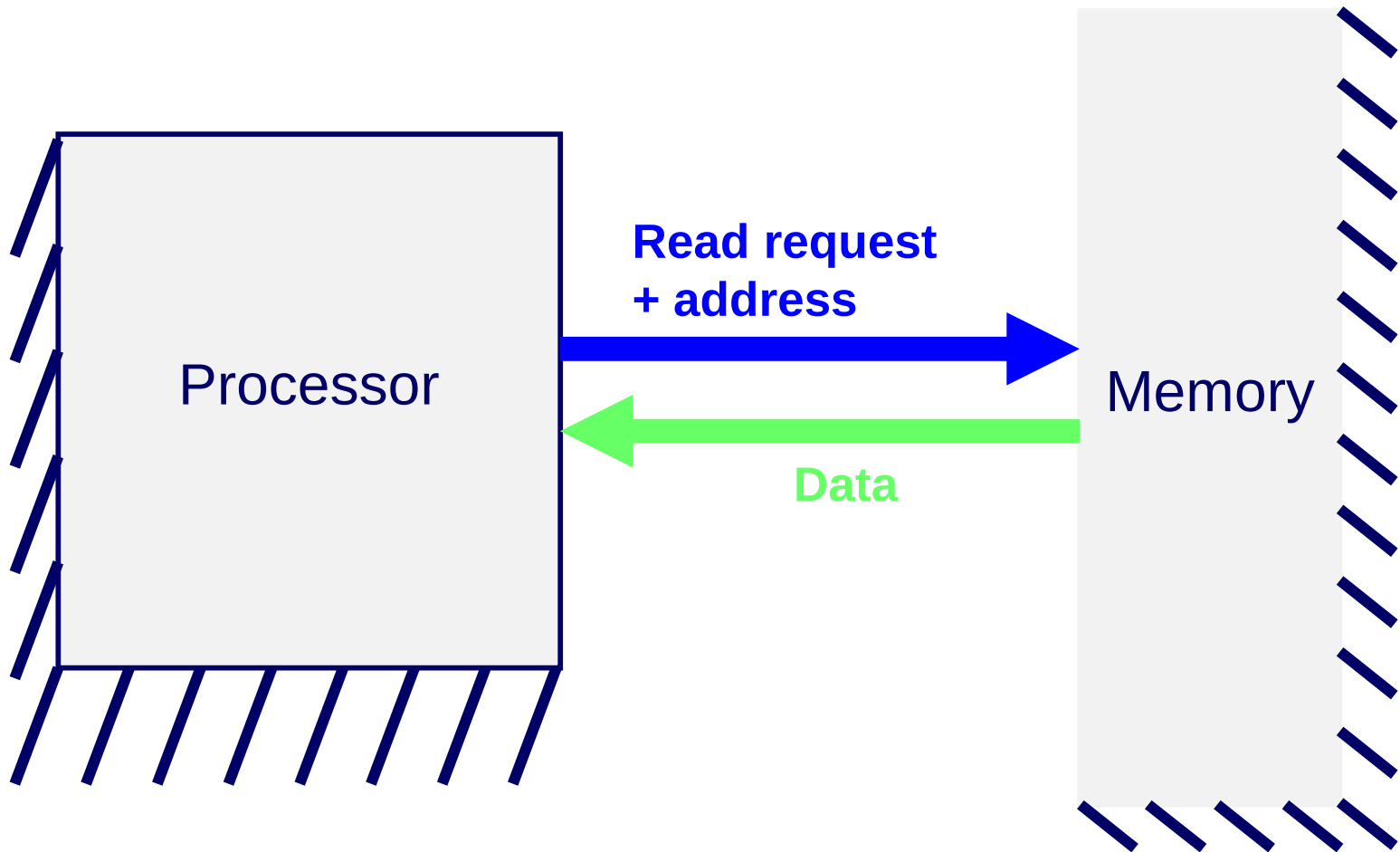
---

- **Store**

- o **Processor sends address** of memory location **with data** to be written
- o Original content of memory at that location is destroyed

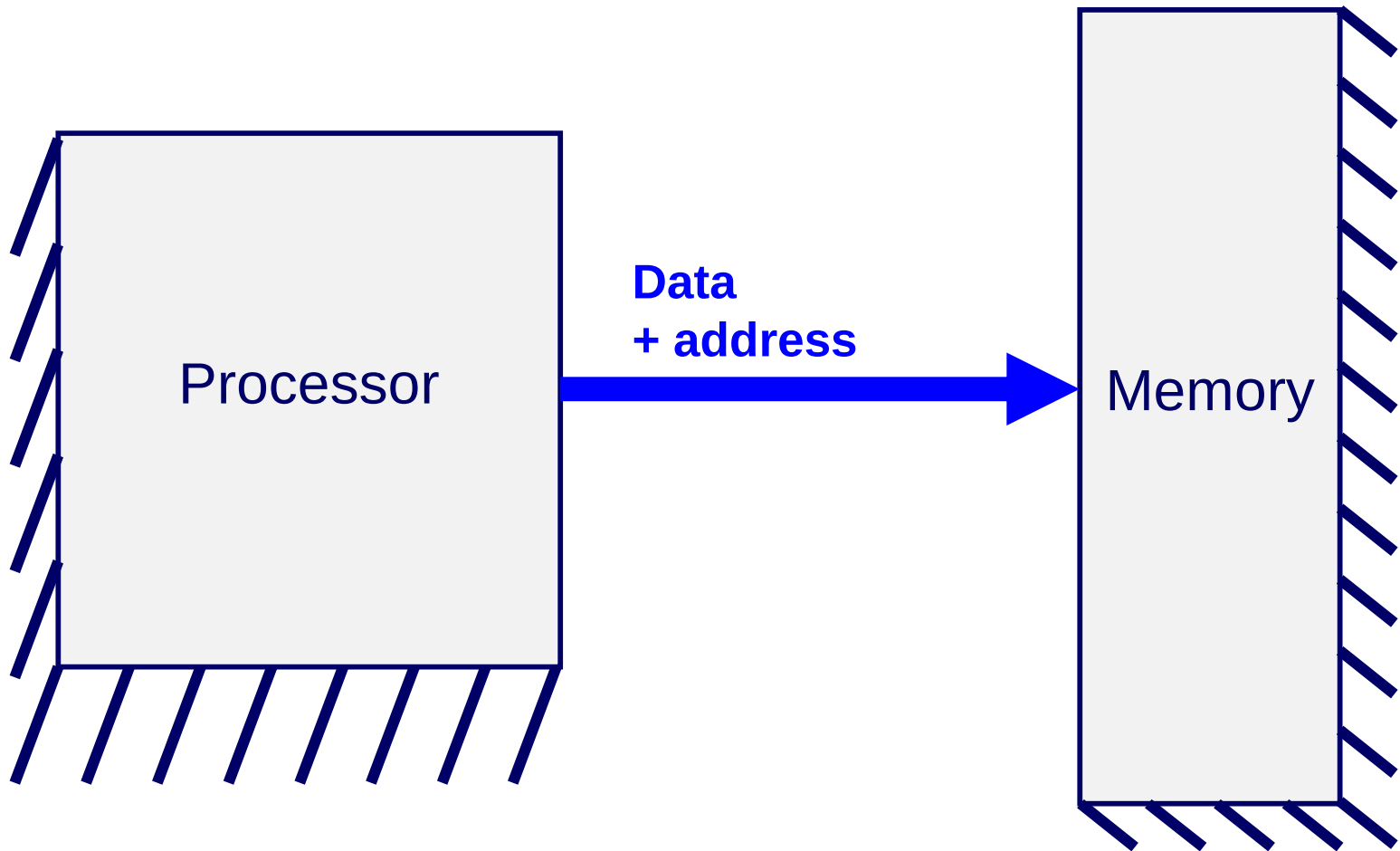
# Memory Operations (Load)

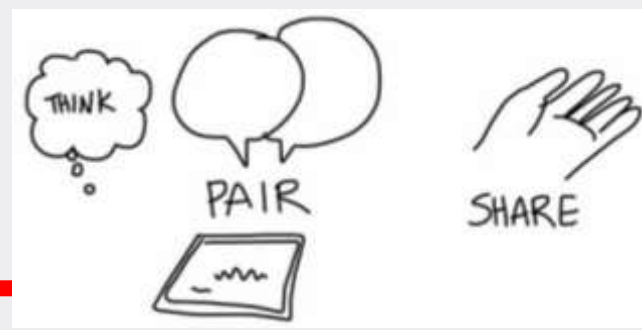
---



# Memory Operations (Store)

---





- What are other operations?

# Outline

---

- Memory locations and addresses
  - Byte addressability
  - Big-endian and Little-endian Assignments
- Memory operations
- Instructions and instruction sequencing
  - Notation
  - Types of instruction
  - Types of operands
  - Types of operations

# Instruction and Instruction Sequencing

---

- Computers must have instructions capable of:
  - o **Data transfer** between memory and processor (registers)
    - Load, Store, Push, Pop, etc.
  - o **Arithmetic and logic** operations
    - ADD, SUB, MUL, DIV, AND, OR, XOR, etc.
  - o Program **sequencing and control**
    - BR, SKP, CALL, etc.
  - o **I/O transfers**
    - Test I/O, Halt I/O, Start I/O, etc.



# But first... some notation

---

- Register Transfer Notation
  - Addresses of memory locations: e.g. LOC, VAR2, A
  - Processor registers: e.g. R0, R1
  - I/O registers: e.g. DATAIN, OUTSTATUS
  - Content of location denoted by square brackets
    - $R1 \leftarrow [LOC]$
    - $R3 \leftarrow [R1] + [R2]$

# But first... some notation

---

- Assembly Language Notation
  - To represent machine instructions
    - Move LOC,R1
    - Add R1,R2,R3

# Outline

---

- Memory locations and addresses
  - Byte addressability
  - Big-endian and Little-endian Assignments
- Memory operations
- Instructions and instruction sequencing
  - Notation
  - Types of instruction
  - Types of operands
  - Types of operations

# Function

- All computer functions are:
  - Data processing
  - Data storage
  - Data movement
  - Control

# Instruction types

---

- Different types of instructions are required to support high-level programming languages.

# Instruction types

---

- **Data processing**
  - Arithmetic instructions process **numerical** data.
  - Logic instructions operate on **bits**.

# Instruction types

---

- **Data storage** (main memory)
  - Memory instructions **move** data between **memory and registers**.

# Instruction types

---

- **Data movement** (I/O)
  - To transfer **programs and data into memory**, and **results back** to the user.



# Instruction types

---

- **Program flow control**
  - Test instructions test the value of a data word or **status** of a computation.
  - Branch instructions **divert the flow** of the program to a different set of instructions depending on the decision made by the test instructions.

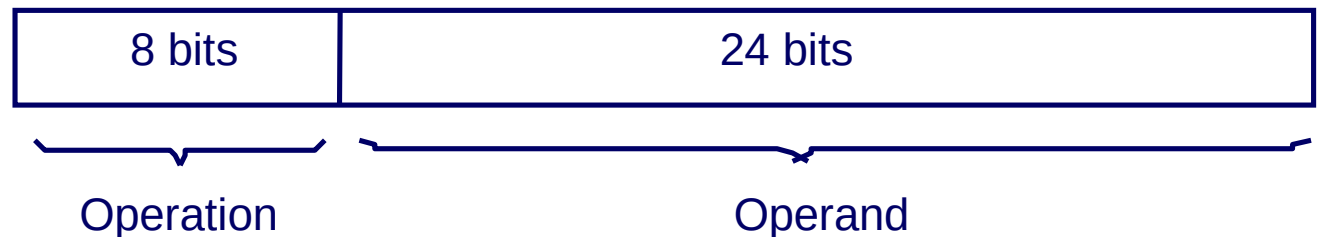
# Attendance

---

# Instruction types

---

- Depends on how many addresses required in the instruction
  - 3-address
  - 2-address
  - 1-address
  - 0-address



# Instruction types

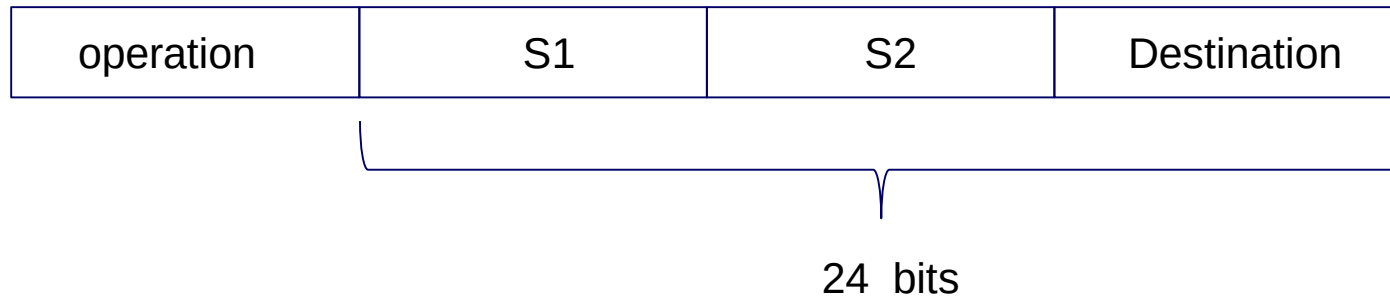
---

- Example:
  - $C = A + B$ 
    - Variable names = memory locations (addresses)
    - Variable values = content of memory location
    - $C \leftarrow [A] + [B]$

# Three-address Instruction

---

- For  $C \leftarrow [A] + [B]$ 
  - Requires 3 memory locations = 3 addresses
  - Add A,B,C
  - Operation Source1, Source2, Destination

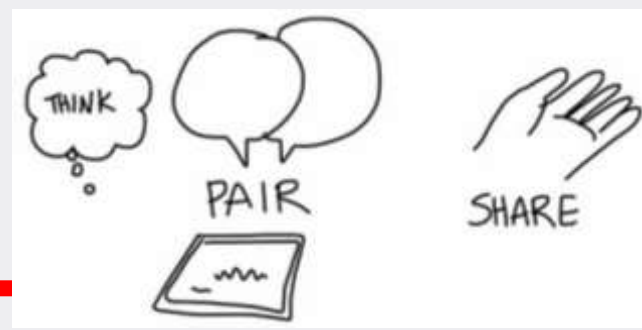


# Three-address Instruction

---

- Considered **too large** to fit in one word
- Format

|           |          |          |             |
|-----------|----------|----------|-------------|
| Operation | Source 1 | Source 2 | Destination |
|-----------|----------|----------|-------------|



- Three-address instruction
- What are the consequences of having 3 addresses in one instruction (+ operation)?



---

Write a program for the following function using three-address instructions:

$$Y = (A + B) * (C + D)$$

ADD R1, A, B     $R1 \leftarrow M[A] + M[B]$

ADD R2, C, D     $R2 \leftarrow M[C] + M[D]$

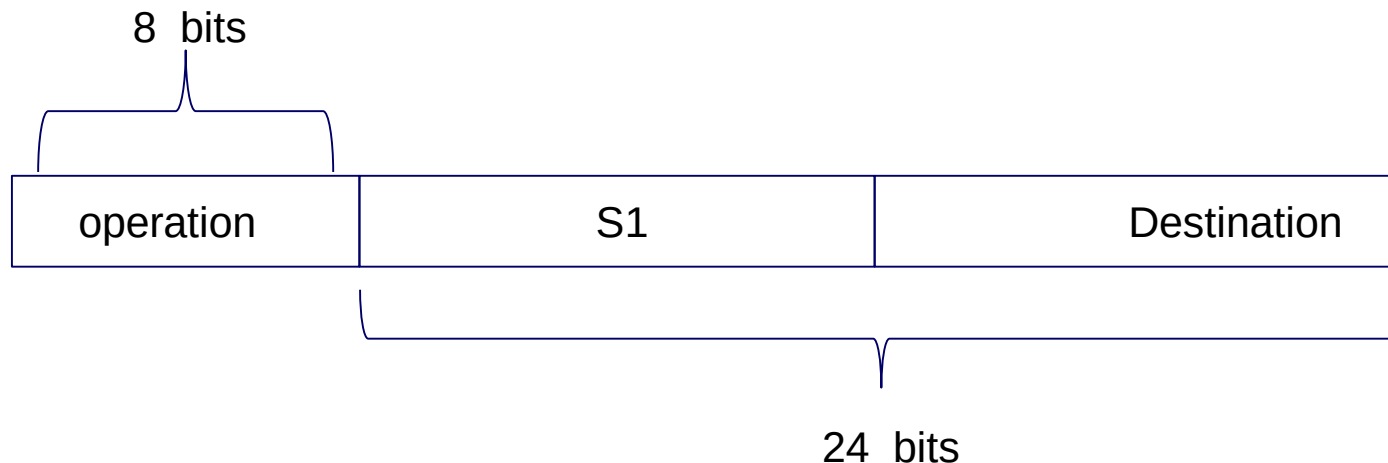
MUL Y, R1, R2     $M[Y] \leftarrow R1 * R2$



# Two-address Instruction

---

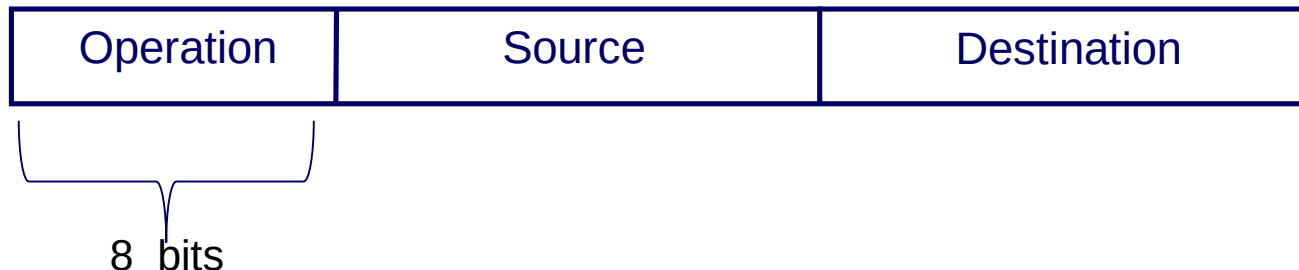
- Simplify  $C \leftarrow [A] + [B]$
- Consider  $B \leftarrow [A] + [B]$ 
  - Requires 2 memory locations = 2 addresses
  - Add A,B
  - Operation Source, Destination



# Two-address Instruction

---

- Original  $C \leftarrow [A] + [B]$  can still be accomplished
  - Move B,C [Move B to C]
  - Add A,C [Add A to C]
- 2-address sometimes **still too long**
- Format



---

Write a program for the following function using two-address instructions:

$$Y = (A + B) * (C + D)$$

MOV R1, A     $R1 \leftarrow M[A]$

ADD R1, B     $R1 \leftarrow R1 + M[B]$  – Perform A + B

MOV R2, C     $R2 \leftarrow M[C]$

ADD R2, D     $R2 \leftarrow R2 + M[D]$  – Perform C + D

MUL R1, R2     $R1 \leftarrow R1 * R2$  (Perform (A+B) \* (C +D))

MOV Y, R1     $M[Y] \leftarrow R1$

# One-address Instruction

---

- When more than one operand is needed, the location of the other operand is **implicit** = **accumulator** (a processor register)

# One-address Instruction

---

- Requires Load and Store (1-address) instructions
- $C \leftarrow [A] + [B]$ 
  - Load A (to accumulator) [Copy content of memory location A into accumulator]
  - Add B (to accumulator) [Add the content of A into the accumulator]
  - Store C (from accumulator) [Store accumulator value to C register]
- Format

| Operation | Source/Destination |
|-----------|--------------------|
|-----------|--------------------|

# Zero-address Instruction and the Stack

---

- Uses the **stack** structure
- Addresses are implied
  - **Stack pointer**

# Stack

---

- List of data elements (words/bytes) with accessing restrictions
- Elements **added/removed at one end only – top** of the stack (other end is bottom)
- LIFO (last in first out)

# Stack

---

- **Push**: place new item on the stack
- **Pop**: remove top item from the stack



# Stack

## Byte Addressability

- Example

- A

- If byte location have addresses 0, 1, 2, ...

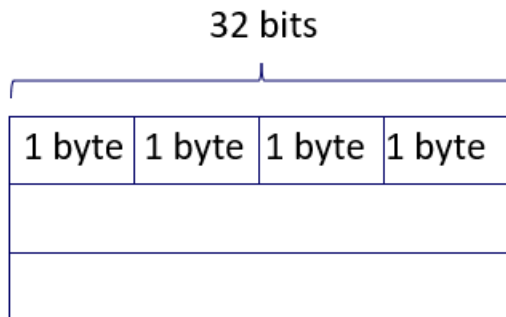
- If word length = 32 bits

- B

- Then successive words are located at addresses 0, 4, 8, ... (each word 4 bytes)

- N

- C



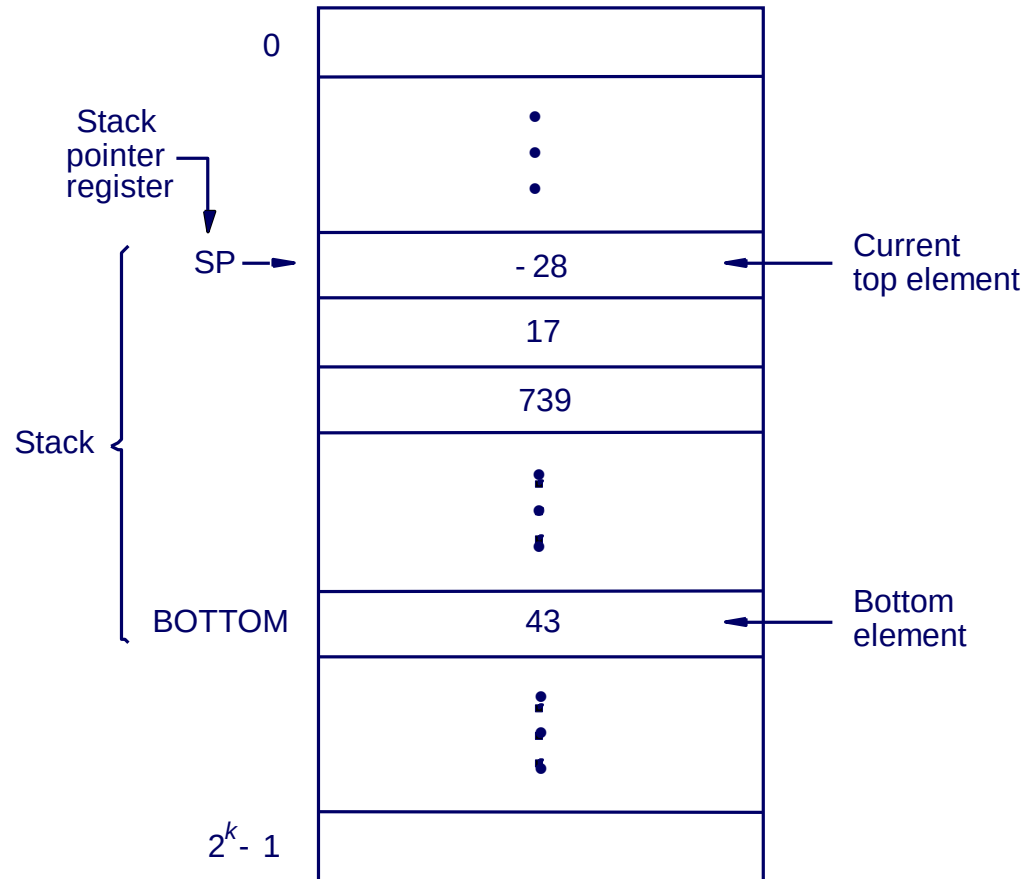
|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 |
| 4 | 4 | 5 | 6 | 7 |
| 8 |   |   |   |   |
|   |   |   |   |   |

ely

- **Stack pointer** (SP) used to keep track of the address of the element at the **top** of the stack

# Stack

---



# Stack

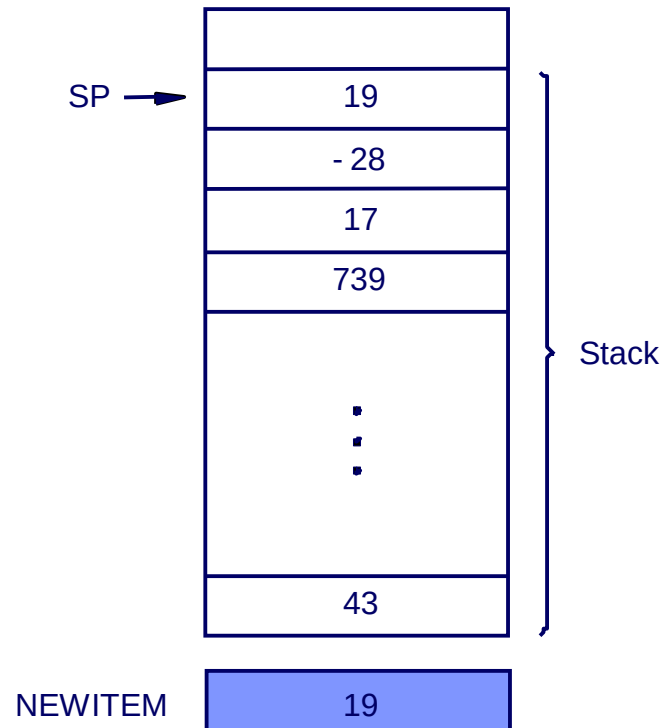
---

- Push

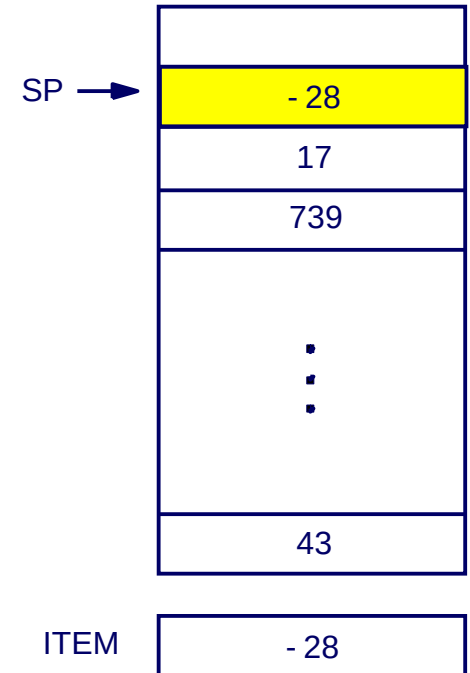
- Subtract #4,SP
- Move  
NEWITEM,(SP)

- Pop

- Move (SP),ITEM
- Add #4,SP



After push from NEWITEM



After pop into ITEM

# Example: $n$ -address instructions on PowerPC

---

$$Y = (A - B) / (C + D * E)$$

## 3-address instruction

|     |         | COMMENT              |
|-----|---------|----------------------|
| SUB | Y, A, B | $Y \leftarrow A - B$ |
| MPY | T, D, E | $T \leftarrow D * E$ |
| ADD | T, T, C | $T \leftarrow T + C$ |
| DIV | Y, Y, T | $Y \leftarrow Y / T$ |

## 2-address instruction

|      |      | COMMENT              |
|------|------|----------------------|
| MOVE | Y, A | $Y \leftarrow A$     |
| SUB  | Y, B | $Y \leftarrow Y - B$ |
| MOVE | T, D | $T \leftarrow D$     |
| MPY  | T, E | $T \leftarrow T * E$ |
| ADD  | T, C | $T \leftarrow T + C$ |
| DIV  | Y, T | $Y \leftarrow Y / T$ |

Note different ordering of source/destination

# Example: $n$ -address instructions on PowerPC

$$Y = (A - B) / (C + D * E)$$

## 1-address instruction

|      |   | COMMENT                |
|------|---|------------------------|
| LOAD | D | $AC \leftarrow D$      |
| MPY  | E | $AC \leftarrow AC * E$ |
| ADD  | C | $AC \leftarrow AC + C$ |
| STOR | Y | $Y \leftarrow AC$      |
| LOAD | A | $AC \leftarrow A$      |
| SUB  | B | $AC \leftarrow AC - B$ |
| DIV  | Y | $AC \leftarrow AC / Y$ |
| STOR | Y | $Y \leftarrow AC$      |

## 0-address instruction

=> Using stack

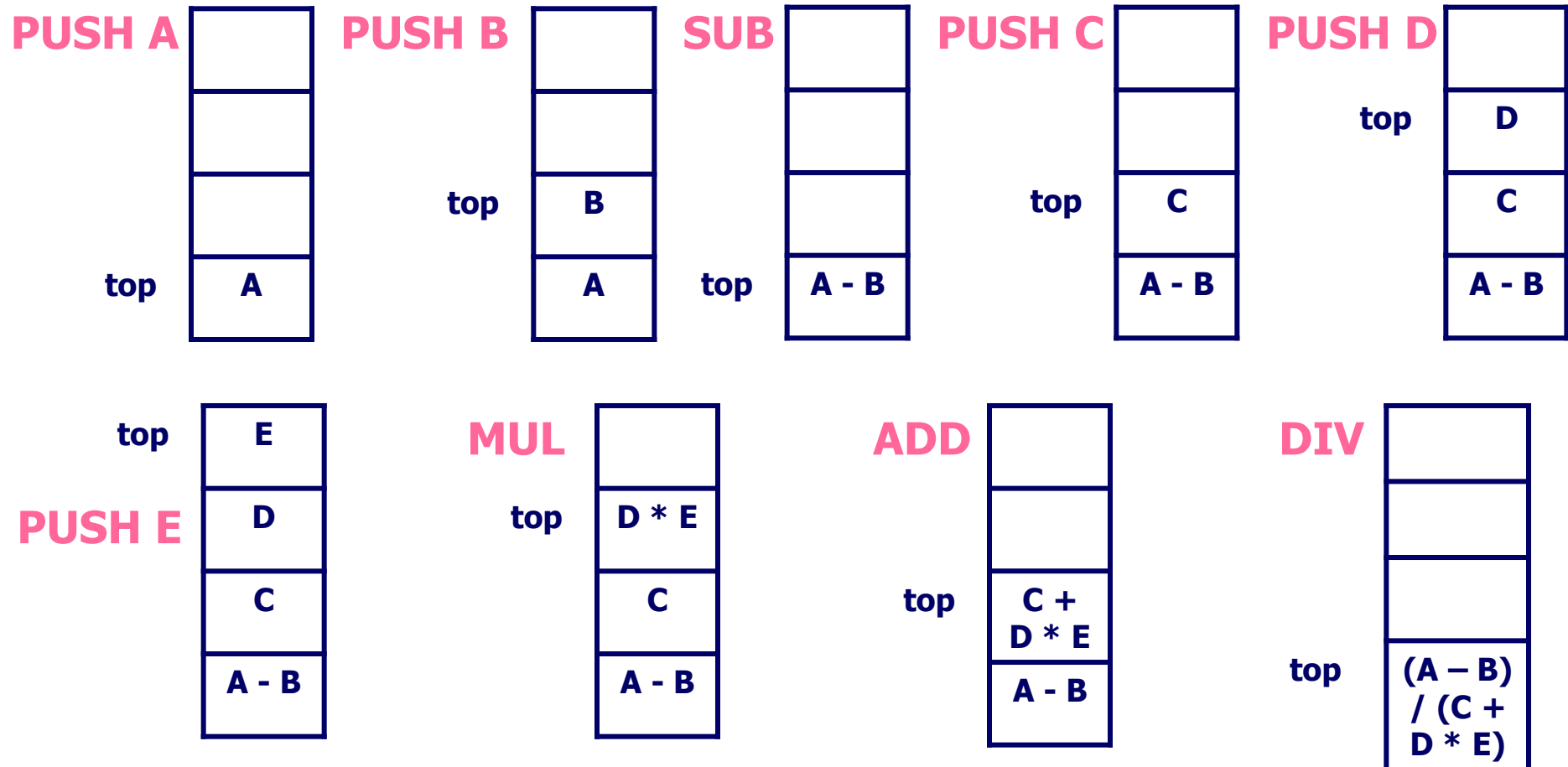
|      |   |                       |
|------|---|-----------------------|
| PUSH | A |                       |
| PUSH | B |                       |
| SUB  |   | A-B                   |
| PUSH | C |                       |
| PUSH | D |                       |
| PUSH | E |                       |
| MUL  |   | D * E                 |
| ADD  |   | (C + D * E)           |
| DIV  |   | (A - B) / (C + D * E) |
| POP  | Y |                       |

Note different ordering of source/destination

# Example: $n$ -address instructions on PowerPC

---

$$Y = (A - B) / (C + D * E)$$



---

$$Y = (A - B) / (C + D * E)$$

| Instruction | Stack                 |
|-------------|-----------------------|
| PUSH A      | A                     |
| PUSH B      | B                     |
| SUB         | (A-B)                 |
| PUSH C      | C                     |
| PUSH D      | D                     |
| PUSH E      | E                     |
| MUL         | (D*E)                 |
| ADD         | (C + D * E)           |
| DIV         | (A - B) / (C + D * E) |
| POP Y       |                       |

# Queue

---

- A data structure
- FIFO (first in first out)
- Assume queue grows in direction of increasing memory addresses
  - New data added at the back (high-memory address)
  - Retrieved from the front (low-address end)



# Queue

---

- Difference between queue and stack
  - One end of stack is fixed (bottom), one pointer for top of stack
  - Both ends of queue move to higher addresses, two pointers for both ends

# How Many Addresses

---

- **More** addresses
  - More **complex** (powerful?) instructions
  - More registers
    - Inter-register operations are quicker
  - **Fewer instructions** per program

# How Many Addresses

---

- **Fewer** addresses
  - Less complex (powerful?), more **primitive** instructions
  - **More (but shorter) instructions** per program
  - **Faster fetch/execution** of instructions

# Design Decisions

---

- **Operation repertoire**
  - How many ops?
  - What can they do?
  - How complex are they?
- **Data types**
- **Instruction formats**
  - Length of op code field
  - Number of addresses

# Design Decisions

---

- **Registers**
  - Number of CPU registers available
  - Which operations can be performed on which registers?
- **Addressing modes** (later...)

# Outline

---

- Memory locations and addresses
  - Byte addressability
  - Big-endian and Little-endian Assignments
- Memory operations
- Instructions and instruction sequencing
  - Notation
  - Types of instruction
  - Types of operands
  - Types of operations

# Types of Operand

---

- Machine instructions operate on data (the operand).
- General categories of data are:
- **Addresses**
  - A form of data.
  - Sometimes, calculations are performed on addresses.
  - Can be considered unsigned integers.

# Types of Operand

---

- Machine instructions operate on data (the operand).
- General categories of data are:
- **Numbers**
  - Integer/floating point/decimal
  - Packed decimal useful for simple decimal computation (0=0000, 1=0001, 2=0010, ..., 9=1001)



# Types of Operand

---

- Machine instructions operate on data (the operand).
- General categories of data are:
- **Characters**
  - ASCII (7-bit pattern = 128 different characters)
  - EBCDIC (8-bit code)

# Types of Operand

---

- Machine instructions operate on data (the operand).
- General categories of data are:
- **Logical** Data
  - Bits or flags

# Outline

---

- Memory locations and addresses
  - Byte addressability
  - Big-endian and Little-endian Assignments
- Memory operations
- Instructions and instruction sequencing
  - Notation
  - Types of instruction
  - Types of operands
  - Types of operations

# Types of Operation

---

- Data Transfer
- Arithmetic
- Logical
- Conversion
- I/O
- System Control
- Transfer of Control

# Data Transfer

---

- Move, Store, Load, Exchange, Clear, Set, Push, Pop
- Specify
  - o **Source** & **destination** (e.g. memory, register, stack, etc.)
  - o **Amount** (length) of data
  - o **Addressing mode**

# Data Transfer

---

- May be different instructions for different movements (between register/memory)
  - e.g. IBM 370
- Or one instruction and different addresses
  - e.g. VAX

# Data Transfer

---

- Data transfer operations is perhaps simplest for CPU:
  - e.g. if source and destination are registers, CPU causes data to be **transferred between registers**

# Data Transfer

---

- Data transfer operations is perhaps simplest for CPU:
  - e.g. if one or both operands are in memory:
    - **Calculate memory address** based on address mode
    - If address in virtual memory, **translate** virtual to actual address
    - Determine address item is in cache
    - If not, issue command to memory module



# Arithmetic

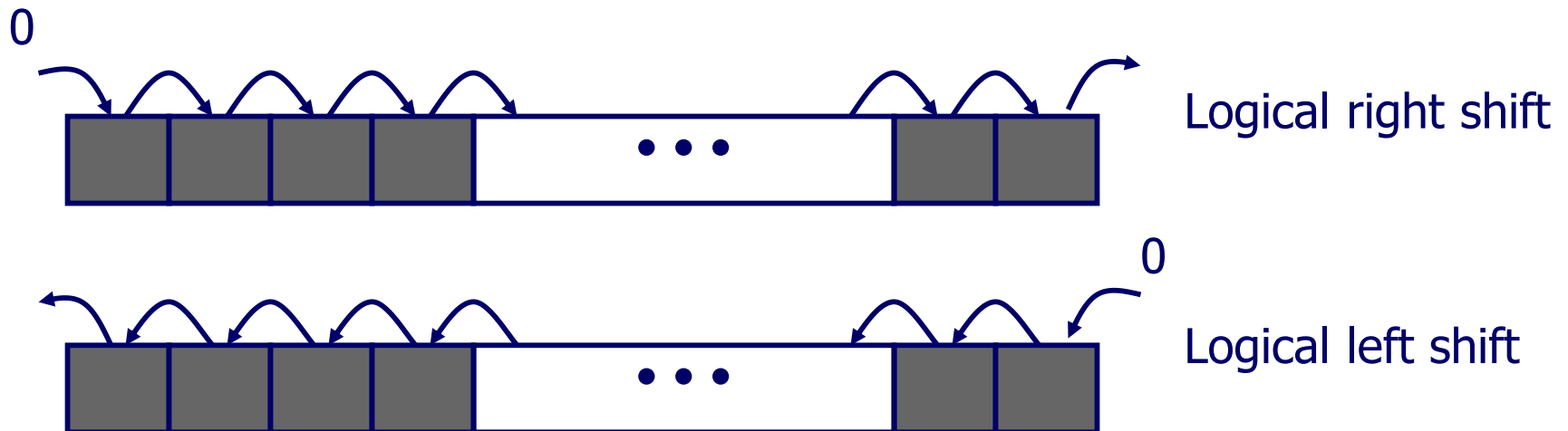
---

- Add, Subtract, Multiply, Divide, Absolute, etc.
- For signed integer (fixed-point) numbers
- Often also for floating point numbers
- May include single-operand instructions:
  - Absolute ( $|a|$ )
  - Increment ( $a++$ )
  - Decrement ( $a--$ )
  - Negate ( $-a$ )
- Involves the ALU

# Logical

---

- Bitwise operations
- AND, OR, NOT, XOR, EQUAL
- Logical Shifts
  - Bits shifted to left or right
  - Bit shifted out is lost; at other end, 0 shifted in



# Logical

---

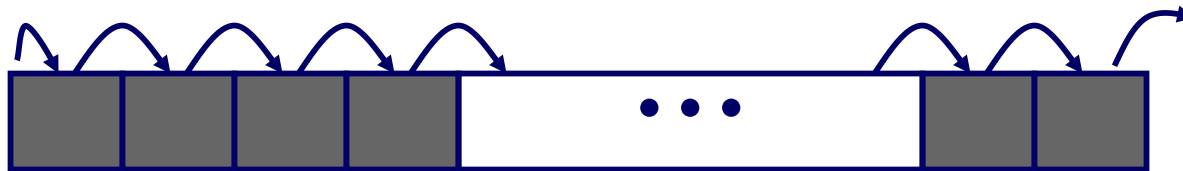
- o Useful to transmitting characters to IO device:
  - e.g. for 16 bit word containing 2 characters
  - Load word into register
  - Shift right eight times (shifts remaining characters to right half of register)
  - Perform IO (IO module read lower-order 8 bits from bus)
    - This is the LEFT character
  - Then load words again
  - AND with 0000000011111111 (masks out character on the left)
  - Perform IO for second character
    - This is the RIGHT character

# Logical

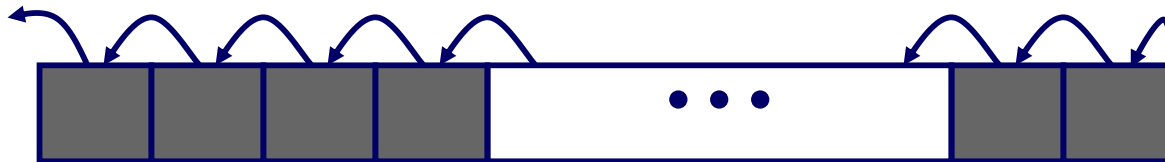
---

- Arithmetic Shifts

- Does not shift sign bit (sign bit is replicated to right or left)



Arithmetic right shift

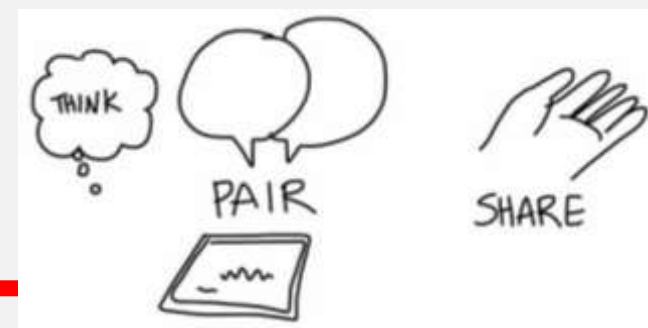


Arithmetic left shift

# Still remember this?!?

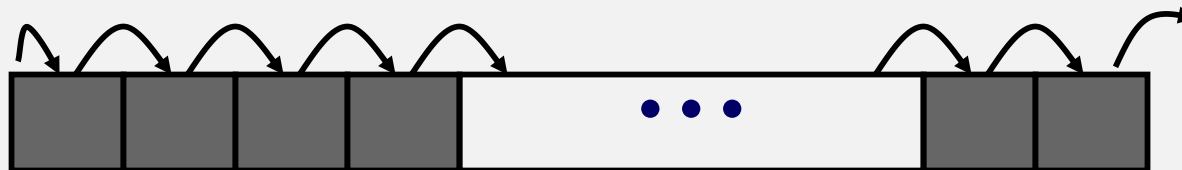
Logical

When do you shift right or left?

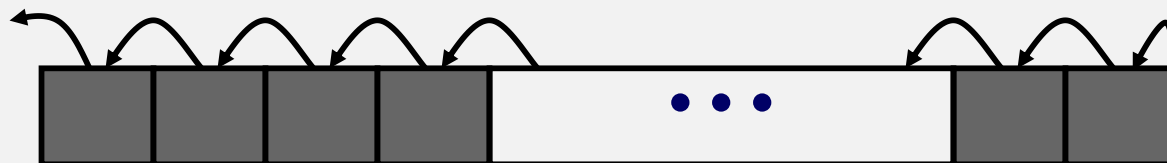


- Arithmetic Shifts

- Does not shift sign bit (sign bit is replicated to right or left)



Arithmetic right shift

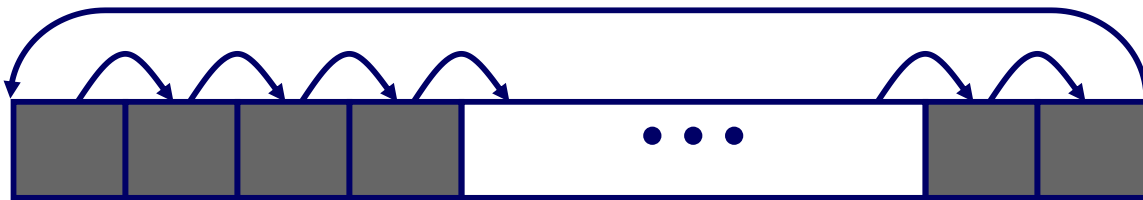


Arithmetic left shift

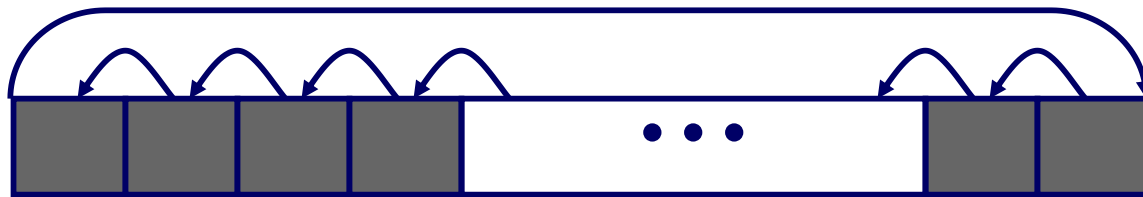
# Logical

---

- Rotate (cyclic shift)



Rotate right



Rotate left

# Conversion

---

- Operations that change the format or operate on the format of data.
- E.g. Binary to Decimal

# Input/Output

---

- May be specific instructions
- May be done using data movement instructions (memory mapped)
- May be done by a separate controller (DMA)



# Systems Control

---

- Privileged instructions
- CPU needs to be in specific state (or program executing in a privileged area of memory)
  - Ring 0 on 80386+
  - Kernel mode
- For operating systems use

# Transfer of Control

---

- Transfer of control operations are required to:
  - execute instructions **more than once**
  - incorporate **decision making**
  - allow programs to be **broken** up into smaller pieces

# Transfer of Control

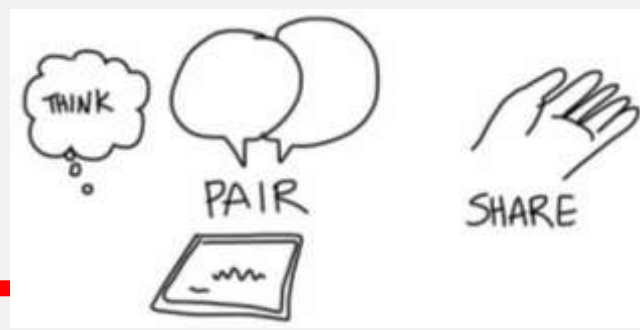
---

- Examples of transfer of control operations:
  - Branch
  - Skip
  - Procedure call
- **Program counter** is updated to contain the address of some instruction in memory

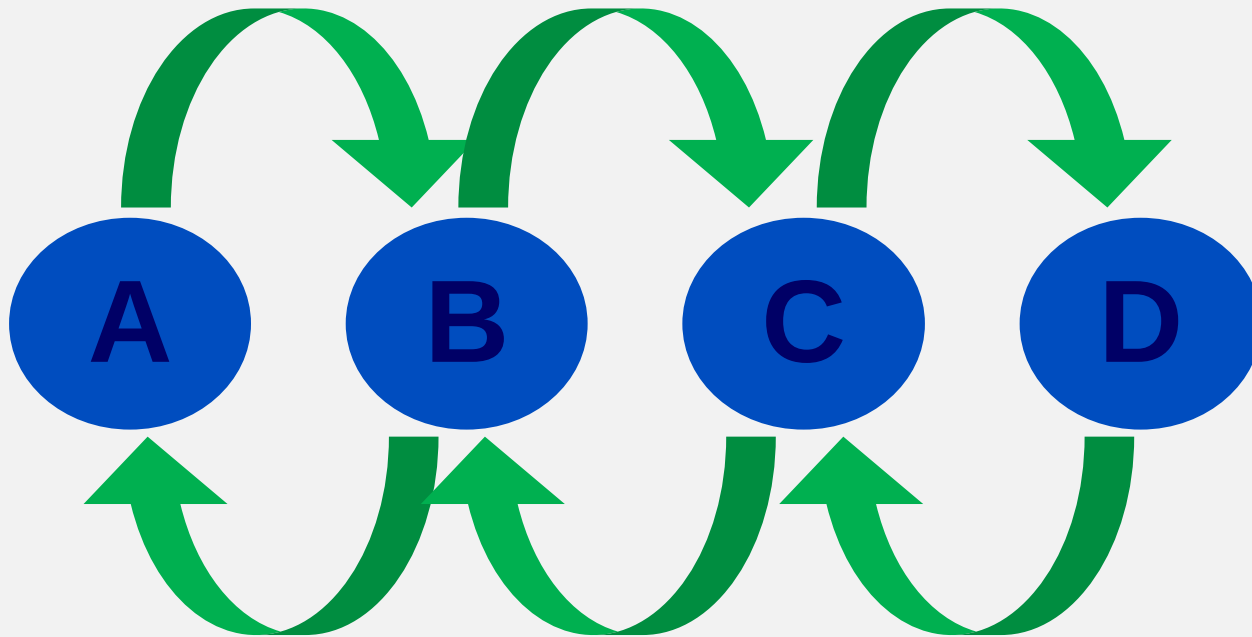
# Transfer of Control

---

- Branch (or jump)
  - e.g. branch to x if result is zero (conditional branch)
- Skip (typically with implied address)
  - e.g. increment and skip if zero
- Procedure or Subroutine call
  - involves a Call instruction from present location to the procedure and a Return instruction from the procedure back to the location from which it was called
  - can involve the use of a stack to keep track of addresses



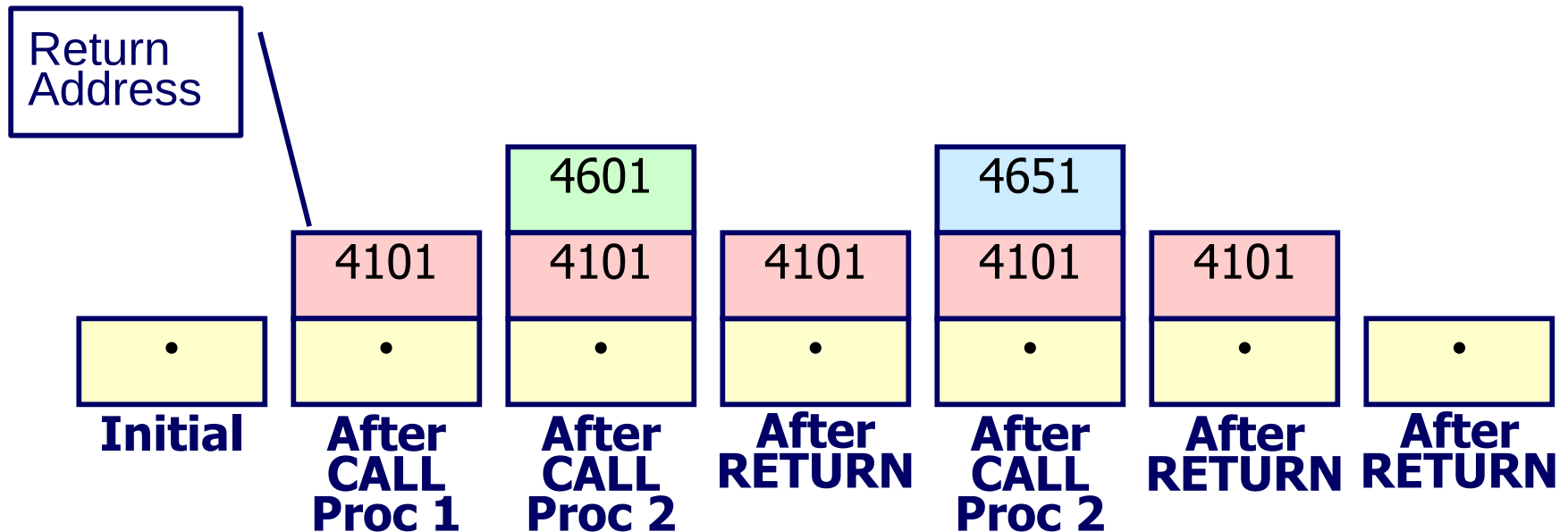
- Which data structure can monitor/keep track of procedure/subroutine calls?



# Transfer of Control

---

- Use of stack to implement nested subroutines (to keep track of return address)



# Straight-line Sequencing

---

- How is program executed
  - **Program counter (PC)** holds address of instruction to be executed next
  - To begin, address of first instruction must be placed in PC
  - Processor uses information in PC to fetch and execute instructions
  - During execution of each instruction, **PC incremented** (e.g. + 4) to point to next instruction

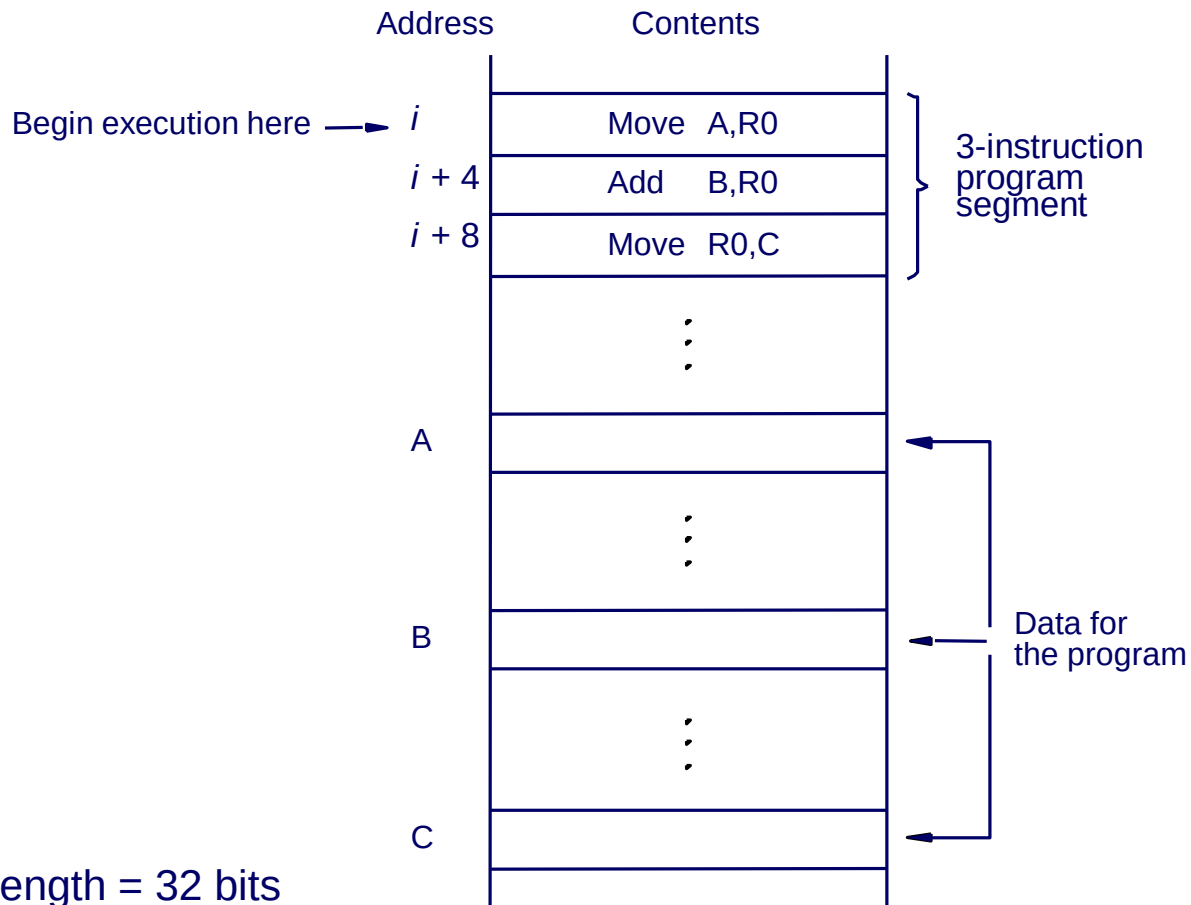
# Straight-line Sequencing

---

- Instruction execution in 2 phases:
  - Phase 1: **Fetch**
    - Fetch instruction based on content of **program counter (PC)**
    - Fetched instruction placed in **instruction register (IR)**
  - Phase 2: **Execution**
    - Content of IR examined to determine which operation to perform
    - Involves fetching operands, performing an arithmetic/logic operation and storing results



# Straight-line Sequencing



Assume

- Word length = 32 bits
- Each instruction = 4 bytes

Program for  $C \leftarrow [A] + [B]$

# Branching

---

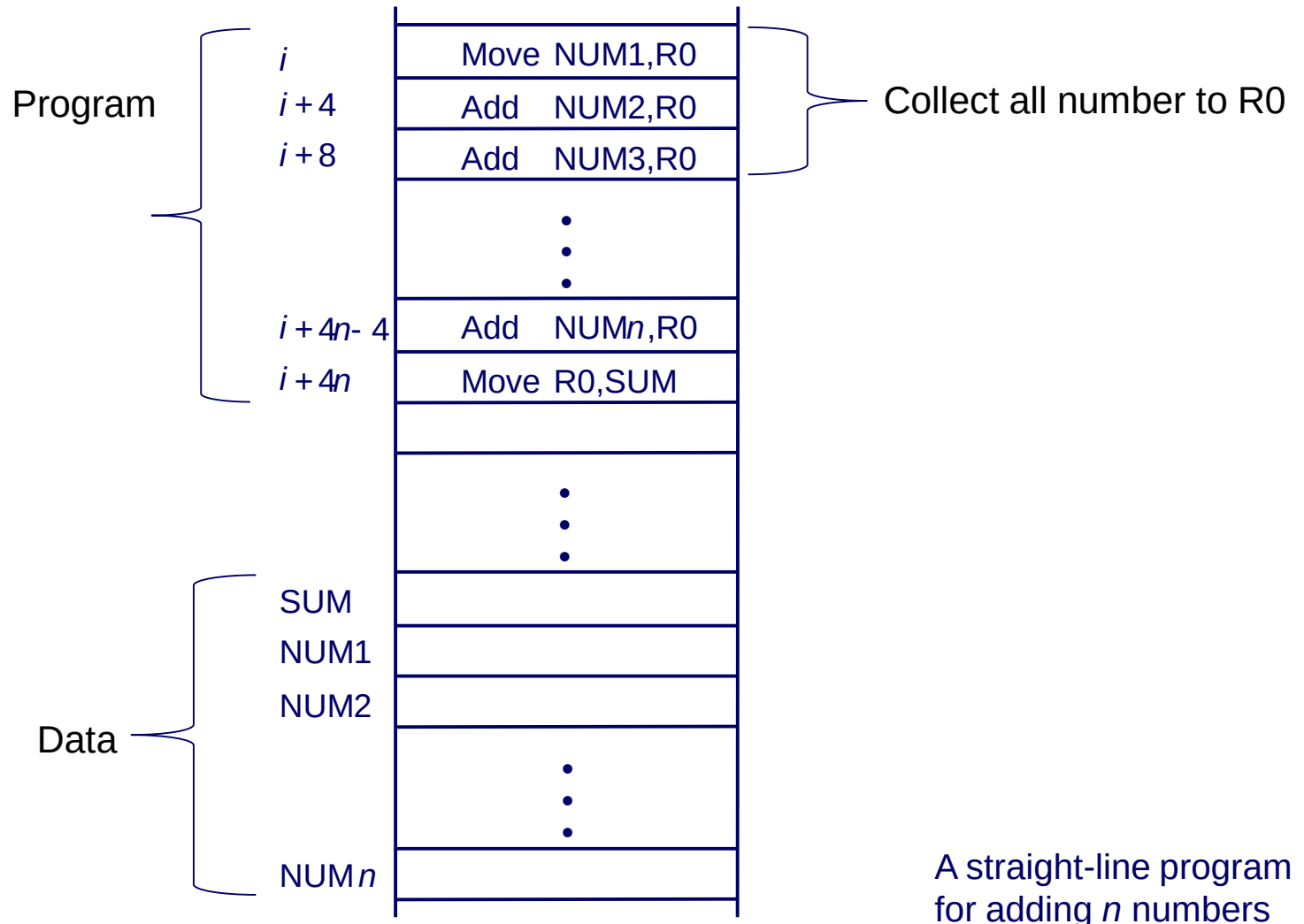
- Instead of using the sequential (incremented from earlier instruction) content of PC, **a new value is loaded into the PC**
- Processor fetches and executes instruction at new address (branch target)

# Branching

---

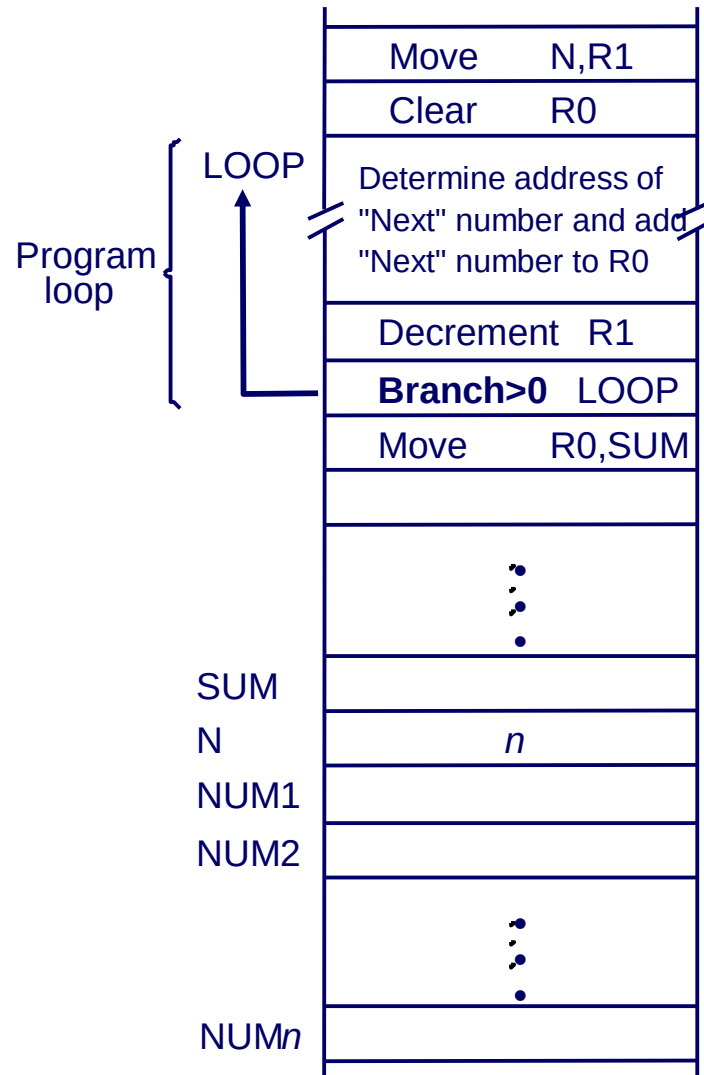
- If a condition needs to be met before causing a branch, it is called a **conditional branch**
- Example
  - Add  $n$  numbers (NUM1, NUM2, ... NUM $n$ ) using conditional branch by checking a control variable stored in R1

# Branching



# Branching

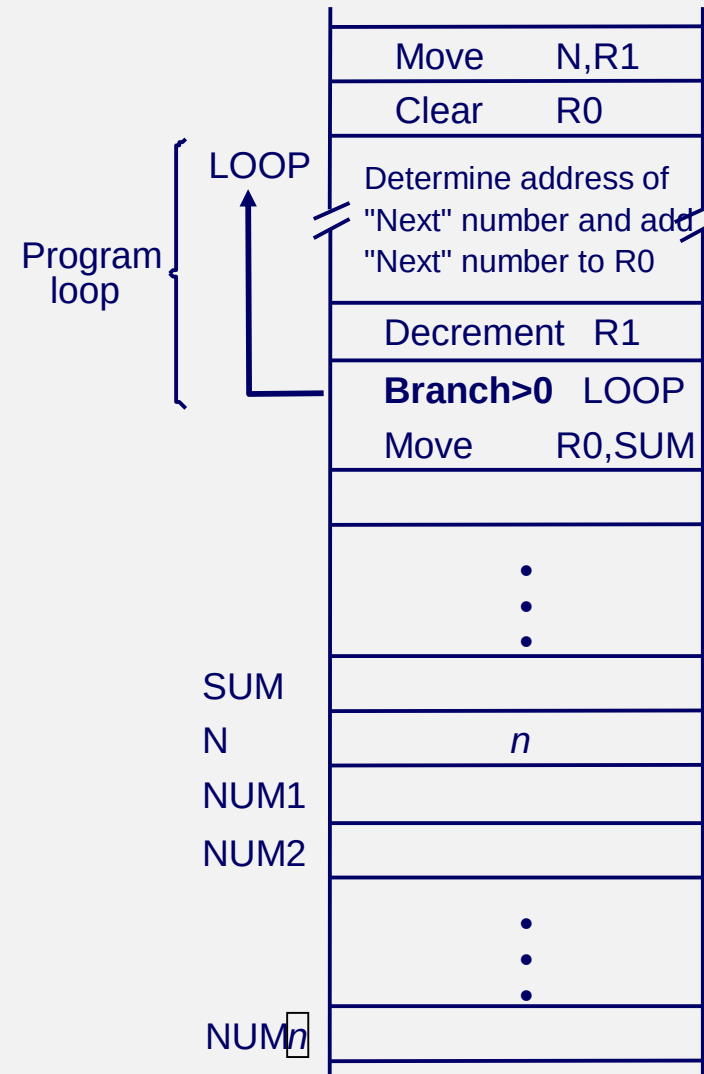
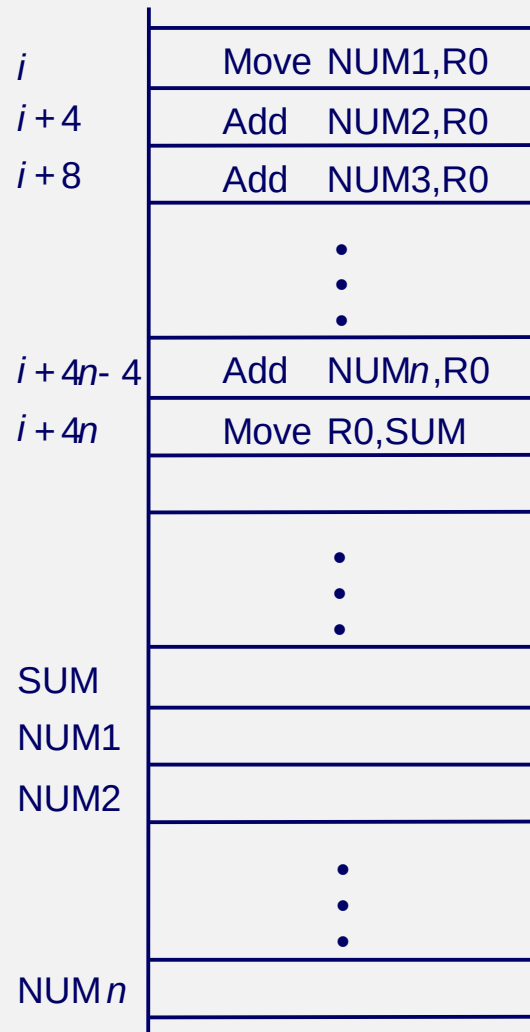
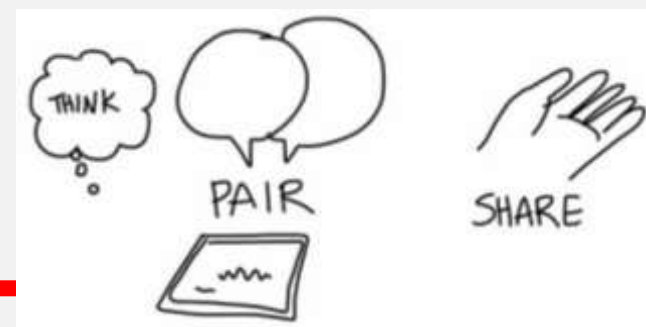
---



Addition happens here

Using a loop to add  $n$  numbers

# No branching vs branching



# Condition Codes

---

- Processor keeps track of information about the results of various operations
- Recorded as bits called condition code **flags**
  - Stored in condition register or status register

# Condition Codes

---

- Common flags
  - N (negative): 1 if result is –ve
  - Z (zero): 1 if result is zero
  - V (overflow): 1 if arithmetic overflow occurs
  - C (carry): 1 if carry-out results from the operation
- Branches may be conditional branches